- If you don't care what an argument is as long as it's there, use `anything`, as in `with("foo", anything)`.
- Any Ruby object that implements triple equal (===) can be used and will match anything it's === to. This will most commonly be regular expressions, such as `with(/\d{3}/)`, but could also be classes or `Proc` objects.
- A hash that includes a particular key can be matched with `with(hash_including(key: value))`.
- Any RSpec matcher can be used.

Many of these arguments are actually RSpec matchers that are being used to match against potential arguments to a double in the same way they match against expectations. You can see a full list in the RSpec documentation.[2]

Mock Tips

My opinion about the best way to use mock objects changes every few months. I'll try some mocks, they'll work well, I'll start using more mocks, they'll start getting in the way, I'll back off, and then I'll think, "Let's try some mocks." This cycle has been going for years, and I have no reason to think it's going to change anytime soon.

That said, some guidelines always hold true.

The Ownership Rule

Don't mock what you don't own. Use test doubles only to replace methods that are part of your application, and not part of an external framework. (Note that you violated this rule in this chapter when you stubbed ActiveRecord methods like `update_attributes`.)

Prescription 22 Don't mock what you don't own.

One reason to mock only methods you control is, well, you control them. One danger in mocking methods is that your mock either doesn't receive or doesn't return a reasonable value from the method being replaced. If the method in question belongs to a third-party framework, the chance that it will change without you knowing increases and thus the test becomes more brittle.

More important, mocking a method you don't own couples your test to the internal details of the third-party framework. By implication, this means the

2. https://relishapp.com/rspec/rspec-mocks/v/3-6/docs/setting-constraints/matching-arguments